

TWiN Term Project Report

Project Title: Intent Driven Autonomous ORAN xApp Development using Agentic AI

Team Members :

Name	Roll No
Mohit Patil	CS24MTECH14013
Harshwardhan Gaikwad	CS25MTECH12004

Abstract

ORAN xApp development is inherently complex and manual, requiring deep domain expertise in both network management and software engineering. This project presents an intent-driven, agentic AI framework that automates the end-to-end development of FlexRIC-compatible xApps from high-level natural language descriptions. Inspired by the AutORAN paper, the system implements a modular multi-agent pipeline orchestrated via LangGraph, consisting of six specialized modules: Intent Decomposer, Technical Mapper, Data Synthesizer or Dataset Profiler, ML Developer, Core Logic Programmer, and FlexRIC Integrator. A shared structured blueprint state is maintained across all modules to ensure consistency, traceability, and reduced hallucination. A Structural RAG system grounded in real FlexRIC codebase is used for accurate telemetry and API mapping. Human-in-the-loop checkpoints are introduced at critical stages to improve correctness. The system supports both synthetic data generation and real dataset ingestion. While core AutORAN features were successfully reproduced and the modular architecture was fully implemented, the code syntax verifier and deployment module remain as future work. Some complex intents still produce faulty xApps, and scalability is limited by sequential LLM inference costs. The project lowers the barrier to xApp development, enabling non-experts to generate deployable network applications from intent alone.

Introduction / Project Recap

Problem statement, motivation, objectives, and project type.

Problem Statement

ORAN xApp development is complex and manual, requiring significant domain expertise and time investment. There is a pressing need for tools that can automate the process of translating high-level network management goals into deployable xApps without requiring deep programming knowledge of the underlying RIC platform.

Motivation

Building a tool that automates xApp development saves significant time and reduces the expertise barrier for network operators. By leveraging Large Language Models and agentic AI frameworks, routine xApp creation tasks can be automated, freeing engineers to focus on higher-level network innovation.

Objective

- (1) Reproduce the AutORAN paper from scratch, including its core intent-to-xApp pipeline.
- (2) Build an end-to-end pipeline that generates working xApps based on natural language intent.
- (3) Use a module-based approach to improve scalability and make the system codebase agnostic, enabling future extension to other RIC platforms beyond FlexRIC.

Project Type -

Engineering Effort, Paper Reproduction

Related Work and Positioning

1. Intent-Driven Network Management with Multi-Agent LLMs (Confucius Framework, SIGCOMM 2025): This paper presents Confucius, a production-ready multi-agent LLM framework that enhances large-scale network management through DAG-based workflows, RAG memory, tool integration, and strong validation mechanisms. It focuses on network management tasks broadly but does not specifically target O-RAN xApp code generation or deployment.

2. AutORAN: LLM-driven Natural Language Programming for Agile xApp Development: AutORAN is an LLM-driven framework that converts high-level user intents into fully deployable O-RAN xApps within minutes, automating the entire development pipeline and significantly accelerating network innovation. It is the primary paper this project reproduces and extends.

Gap Addressed: This project reproduces the core features of AutORAN by enabling modular, blueprint-driven, and FlexRIC-grounded generation of deployable xApps from intent. Compared to prior LLM-based systems, it improves reliability and end-to-end practicality through a shared structured blueprint state, Structural RAG grounded in real FlexRIC code, human-in-the-loop checkpoints, and a clearly separated six-module pipeline. The design is also intentionally made codebase-agnostic for future extensibility.

System Model / Threat Model

System Model: The system implements a multi-agent pipeline covering the full xApp lifecycle: intent decomposition, technical mapping, data or ML handling, core logic programming, and FlexRIC integration. LLM agents are orchestrated via LangGraph with a shared structured blueprint state maintained across all modules. A Structural RAG system indexes the real FlexRIC codebase to ground technical mappings. Workspace tools handle code generation and execution. An optional ML training and data pipeline supports both synthetic and real datasets. The final output is a deployable FlexRIC xApp (`final_xapp.py`).

Key Assumptions:

- (1) LLM outputs are reliable when constrained by structured prompts and RAG grounding.
- (2) The user provides a coherent, actionable network management intent.
- (3) The FlexRIC codebase is available locally for RAG indexing.
- (4) The pipeline runs in a controlled workspace environment where generated code can be safely executed and validated on small data chunks.

System Architecture and Methodology

Include architecture diagram, modules, and data/control flow.

Core Algorithm / Design Logic

The system follows a structured, decision-driven pipeline where each stage refines the output before passing it forward. It begins by converting user intent into a structured blueprint via Module 1 (Intent Decomposer), which is then validated by a human reviewer to ensure correctness. The system grounds this intent into concrete technical mappings using Module 2 (Technical Mapper), which leverages the FlexRIC Structural RAG for accurate API and telemetry mapping. At the data stage, a key decision is made: either synthesize data via Module 3a (Data Synthesizer) or profile an uploaded real dataset via Module 3b (Dataset Profiler). The pipeline then conditionally branches into Module 4 (ML Developer), triggered only if a machine learning model is required by the intent. The resulting data or model feeds into Module 5 (Core Logic Programmer), which produces the core decision-making function. Finally, Module 6 (FlexRIC Integrator) assembles all components into a deployable `final_xapp.py`. Decisions at critical checkpoints (human validation, dataset choice, ML necessity) ensure both flexibility and reliability throughout the workflow. The shared blueprint JSON state is updated and passed through each module, serving as the single

source of truth for the entire pipeline.

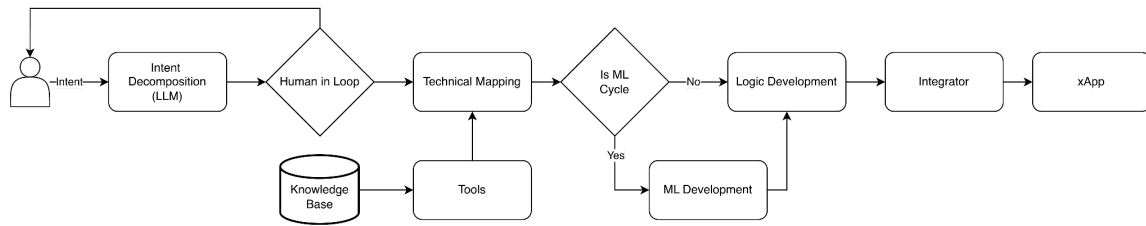


Fig. System Architecture

Implementation Details

Hardware Setup:

Local system for orchestration and development; Lab server with GPUs for local LLM deployment via Ollama.

Software and Tools:

LangChain and LangGraph for agent orchestration and pipeline execution; Ollama and OpenAI as LLM backbones; FAISS and ChromaDB for Dual-RAG indexing; Python for core development; Docker for containerization; FlexRIC as the target RAN platform; OpenAirInterface (OAI) for radio environment simulation.

What was Built vs Reused:

Reused and extended components include LangChain/LangGraph for orchestration and frontend (LangSmith + LangChain UI), OrioSearch as a web-search fallback enhanced for O-RAN queries, and a Semantic Search service later complemented by the Structural RAG approach. Built from scratch: Modules 1 through 6 forming the complete end-to-end agent pipeline, FlexRIC Structural RAG as a custom code-aware retrieval system, the blueprint-driven shared state architecture, and a Terminal Command Runner extending LangChain's file manager for safe code execution in a controlled workspace.

Key Challenges and Solutions:

- (1) Hallucinations in technical mapping were mitigated using Structural RAG grounded in real FlexRIC code. - This challenge still persists as it is in inherent nature of LLMs to hallucinate
- (2) Mismatch between natural language and system-level variables was addressed via a two-step mapping (intent to blueprint to technical mapping). - - This challenge still persists as the small scale llms do not have context to codebases such as that of FlexRIC and RAGs do not solve the problem completely
- (3) Unreliable data generation and schema misalignment were fixed by standardizing Data_Paths contracts with schema validation.
- (4) Context explosion and tool-calling loops were controlled through human-in-the-loop

checkpoints and tool call limits.

(5) FlexRIC integration complexity was managed by separating core logic (Module 5) from deployment integration (Module 6).

(6) End-to-end reliability was ensured through modular pipeline contracts, allowing failures to be traced and fixed at specific stages.

GitHub Repository:

<https://github.com/mohitpatil30501/Intent-Driven-Autonomous-ORAN-xApp-Development-using-Agentic-AI>

Results and Analysis

We tested our implementation against two other models - Gemma e4b/4b and Gemini 3.1 Fast

Aggregated Scores per Model per Intent

Intent	Max	Gemma (e4b/8b)	Gemini 3.1 Fast	AutORAN
SliceDetailMonitor	40	9 (22.5%)	11 (27.5%)	35 (87.5%)
PRBMonitor	40	9 (22.5%)	11 (27.5%)	25 (62.5%)
UEClassifier	50	15 (30%)	11 (22%)	39 (78%)
UETrafficLoadBalancer	40	9 (22.5%)	11 (27.5%)	27 (67.5%)
AnomalyDetection	50	17 (34%)	14 (28%)	39 (78%)
Aggregate	220	59 (26.8%)	58 (26.4%)	165 (75.0%)

Per-criterion average across all 5 intents (normalized to /5)

Criterion	Gemma	Gemini	AutORAN
C1 — API Fidelity	0.0	0.0	3.4
C2 — Build Success	0.0	0.0	3.4
C3 — E2SM Subscription	0.0	0.0	4.6

C4 — Intent Coverage	3.0	3.0	4.2
C5 — ML Suitability (where applicable)	4.0	1.5	5.0
C6 — ML Artifact Readiness (where applicable)	3.0	0.0	5.0
C7 — Structural Soundness	0.0	0.0	5.0
C9 — Hallucination Inverse	0.0	0.0	3.4
C10 — Generation Latency	3.0	5.0	0.0
C11/C12 — Readability	3.0	3.0	5.0

Rubric -

1. Platform Integration & API Fidelity (Highest Weight)

Evaluates how accurately the code interacts with the FlexRIC SDK. It measures the use of genuine APIs and C-structs (C1), the ability of the code to compile and build without manual patching (C2), the correctness of E2 Service Model (E2SM) subscriptions (C3), and strictly penalizes AI-generated hallucinations of non-existent libraries or endpoints (C9).

2. Functional Correctness & Intent

Assesses whether the generated xApp actually solves the requested problem. It measures how well the explicit and implicit requirements of the user's prompt are fulfilled (C4) and ensures the xApp logic executes deterministically and reproducibly across multiple runs (C8).

3. Machine Learning Suitability & Readiness

For xApps requiring AI/ML, this evaluates the selection of the ML model family and its input/output shapes (C5). It also checks for strict adherence to MLOps best practices, ensuring ML models are treated as loadable offline artifacts rather than being trained dynamically at runtime (C6).

4. Software Engineering Quality

Focuses on the structural and maintainable aspects of the code. This includes adherence to standard FlexRIC project template structures and separation of

concerns (C7), overall code readability, commenting, and maintainability (C11), as well as the end-to-end latency of the generation process (C10).

Conclusions

1. AutORAN dominates on platform-correctness criteria. It is the only system scoring non-zero on the FlexRIC-binding criteria (C1, C2, C3, C7, C9). Gemma and Gemini both flatline at 0 on every platform integration metric — neither produces deployable xApp code against FlexRIC.
2. Gemma and Gemini fail in the same way for different reasons. Gemma never tries to integrate (pure mock simulations); Gemini tries but hallucinates the wrong framework (ricxappframe from OSC, not FlexRIC's xapp_sdk). Their aggregate scores are nearly identical (~27%) — both essentially produce non-functional xApps.
3. AutORAN's weakness is C-struct field hallucination, not architecture. On 4 of 5 intents it loses points on C1/C2/C9 due to ~1–2 hallucinated struct properties per 100 LOC (e.g., kpi_indication_message instead of ue_stats, fabricated control fields). These are described as " ≤ 3 trivial fixes" — i.e., near-deployable rather than fundamentally broken.
4. AutORAN has one substantive logic bug. The UETrafficLoadBalancer drops to 67.5% because core_logic.py hardcodes an 85% threshold (vs. the 80% the intent specified) and ignores the UE-count clause entirely. This is the only case where it misreads the explicit intent rather than just mis-binding C structs.
5. The latency tradeoff (C10) is real. AutORAN scores 0 on generation latency across the board, while Gemini gets a perfect 5. AutORAN is paying multi-agent orchestration cost to gain ~50 percentage points of correctness.
6. ML lifecycle handling is the clearest qualitative gap. On the two ML intents, AutORAN is the only system that treats models as pre-trained external artifacts with graceful fallbacks. Gemini retrains inside `__init__` on every launch (and even refits unsupervised models per micro-batch in AnomalyDetection), which is architecturally wrong for edge inference.

Bottom line: AutORAN is the only candidate producing FlexRIC-deployable code. Gemma/Gemini outputs would need to be discarded and rewritten; AutORAN outputs would need ≤ 3 line edits per file plus one threshold-logic fix.

Limitations and Failure Analysis

Where the System Breaks:

- (1) The pipeline struggles with moderate to complex intents involving multiple telemetry metrics and interdependent actions.
- (2) Errors occur when mapping C-based structs and extracting exact telemetry fields during

code or RAG search.

(3) Occasional hallucination of non-existent technical mappings when converting natural language to technical specifications.

(4) Ambiguous intents can trap the LLM in repeated tool-calling loops, inflating context and degrading performance.

(5) Failures in data synthesis occur when aligning generated dataset columns with actual telemetry variable names.

(6) Unexpected or redundant tool calls lead to inefficiency and unstable execution flow.

Unrealistic Assumptions:

Assuming the NLP to structure intent to technical mapping chain is error-free is unrealistic. Ambiguity and missing context in user intents frequently affect the accuracy of downstream modules.

Scalability Issues:

High latency and cost arise from multiple sequential LLM calls across modules. Context growth from multi-step reasoning and tool interactions can exceed model context limits or reduce model effectiveness for long pipelines.

Evaluation Against Objectives

Objective 1: Reproduce the AutORAN Paper from scratch. Status: Partially Completed.

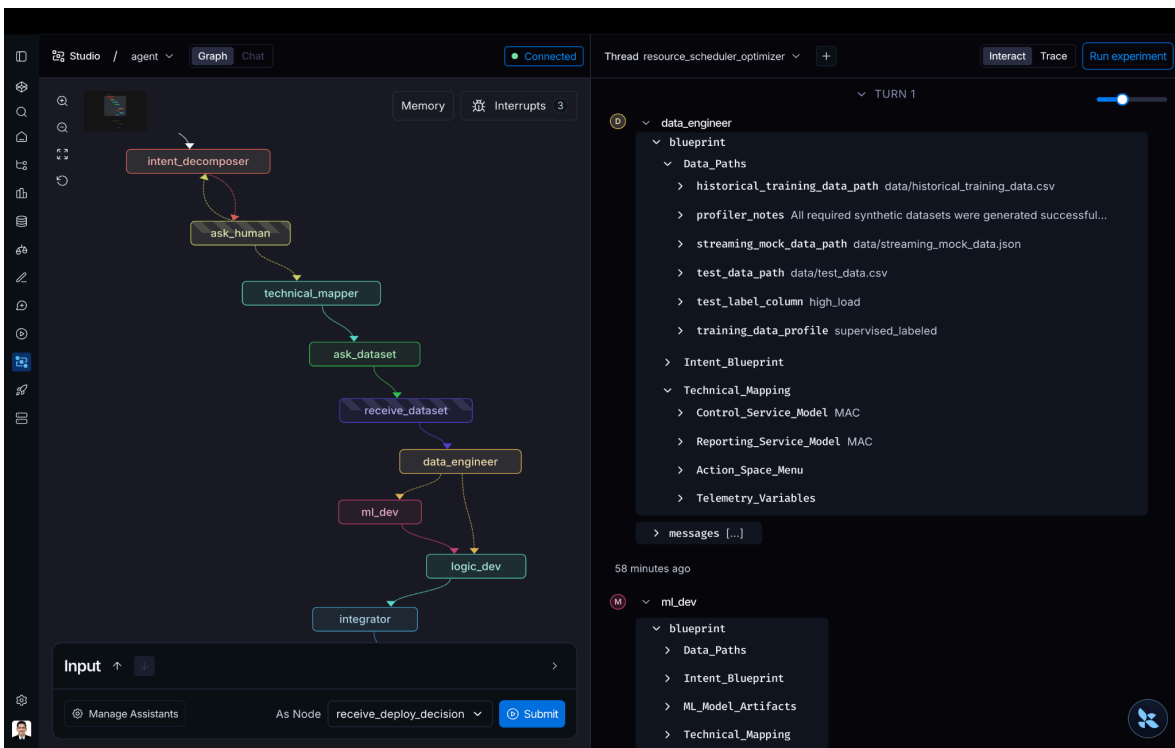
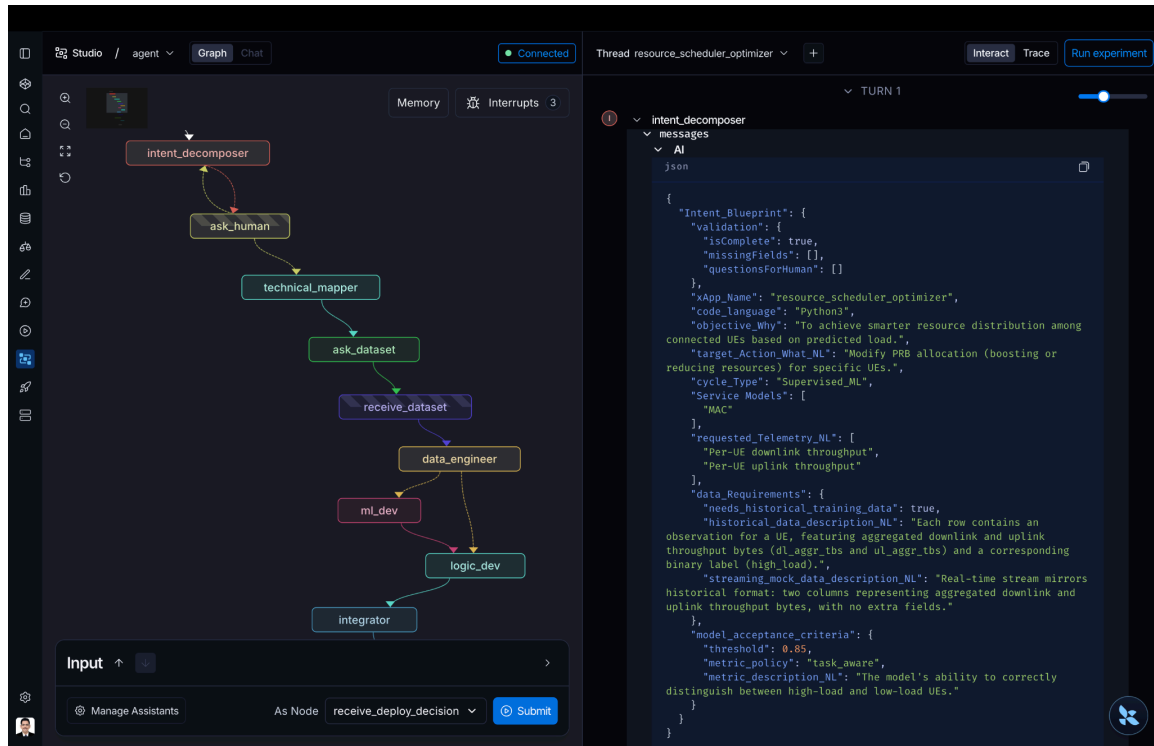
The core intent-to-xApp pipeline was reproduced with six functional modules. The code syntax verifier and full deployment module remain as future work. Evidence: Modules 1 through 6 are implemented and functional for standard intents.

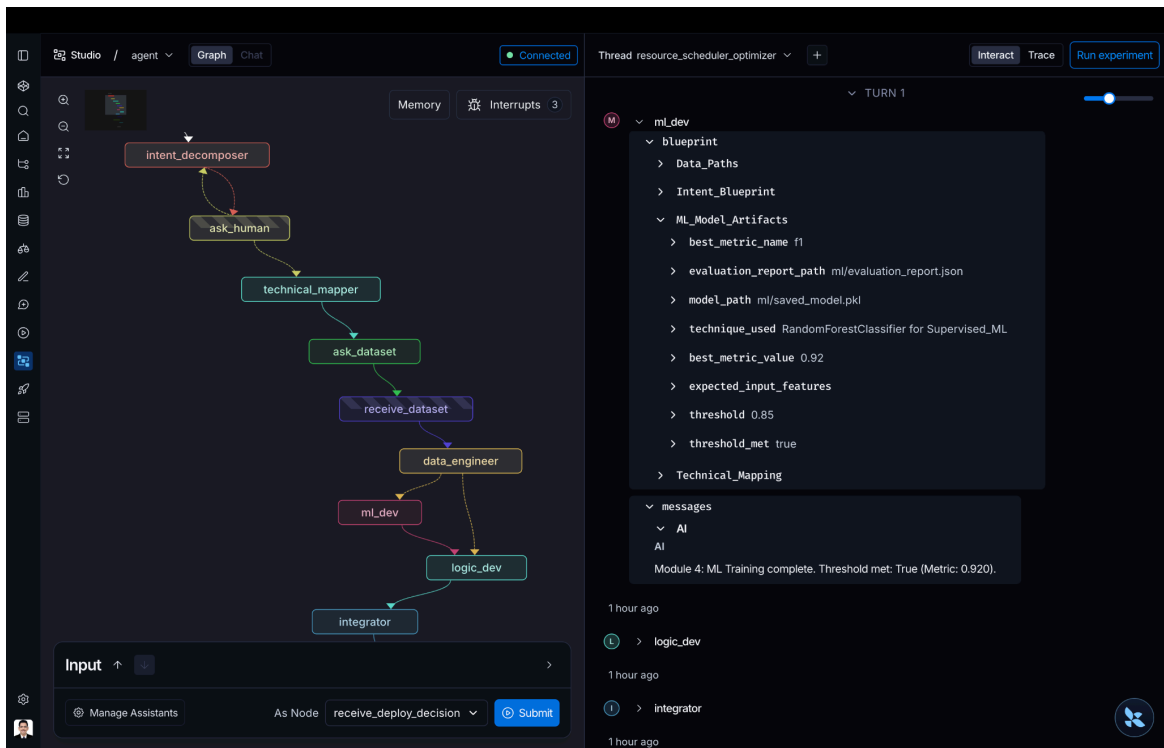
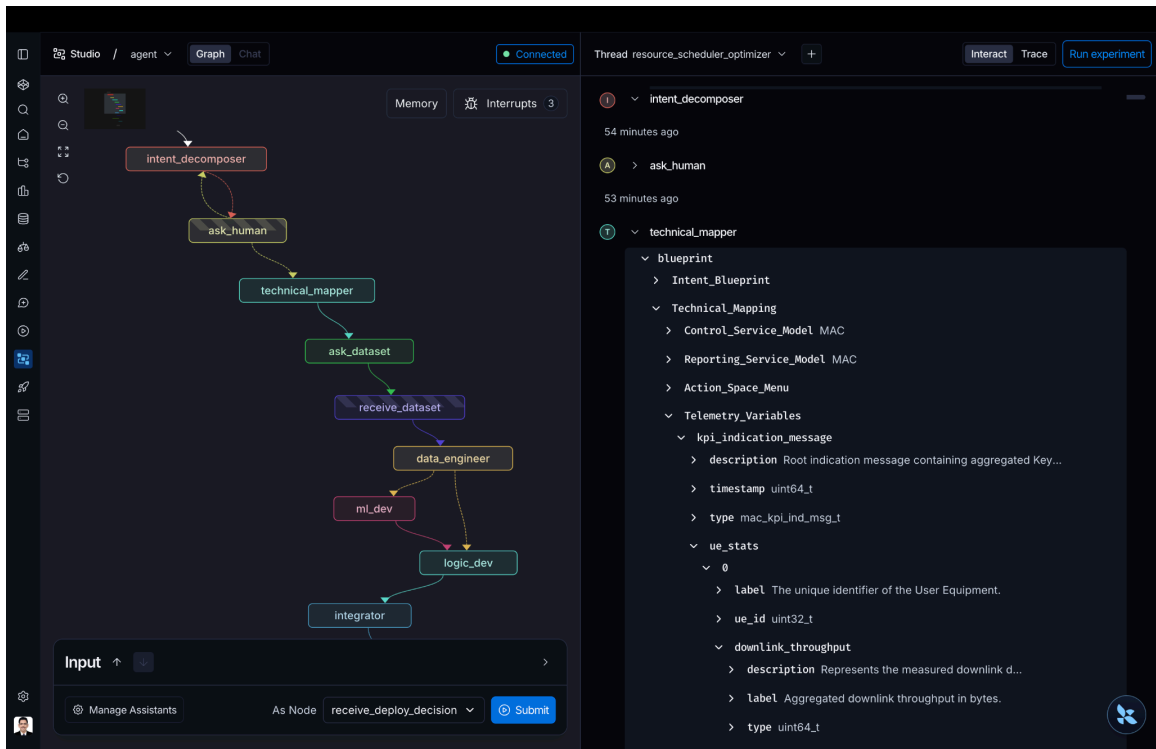
Objective 2: Build an end-to-end pipeline that generates working xApps from natural language intent. Status: Partially Completed. The pipeline successfully generates syntactically and logically correct xApps for simple and moderately complex intents. Some complex intents still produce faulty xApps. Evidence: Successful xApp generation demonstrated for monitoring, orchestration, and optimization intents.

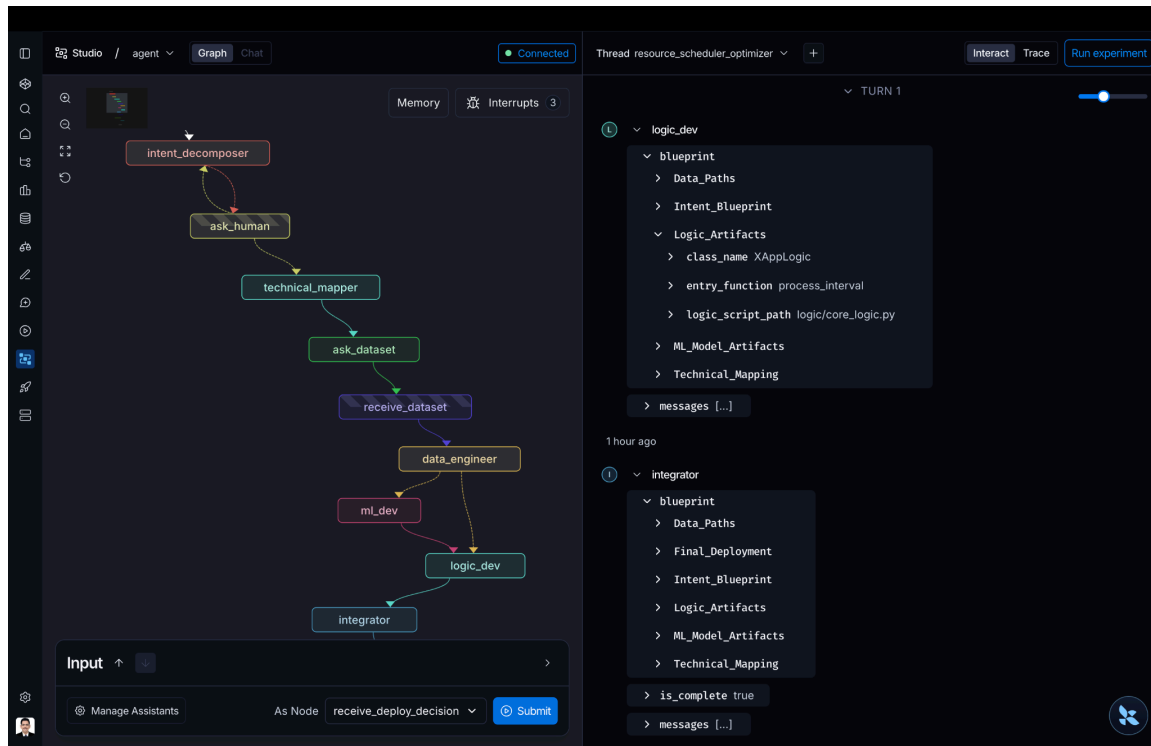
Objective 3: Use a module-based approach to improve scalability and make the system codebase agnostic. Status: Completed. Six clearly separated modules with defined contracts were implemented. The design is extensible to other RIC platforms. Evidence: Each module operates independently with shared blueprint state, and the RAG subsystem is designed to be swappable per platform.

Objective 4: Beat Spotlight Dataset. Status: Infeasible. This objective was determined to be infeasible within the project scope and timeline.

Demo Description







This describes the end to end process of from NL intent to Final xApp delivery

Work Distribution

Name	Core Responsibility	Key Tasks & Contributions
Mohit Patil	Design, Planning and Execution	Modules 1-7 Scaffoldings and Initial implementation, Tools developments and expansion and Langchain Deployment, Logic Developer (Module 5), Integrator Development (Module 6), Deployer (Module 7)
Harshwardhan	Design and Execution	FlexRIC Structural RAG, Mapper (Module 2), Data Synthesizer (Module 3) and ML Module (Module 4) with Feature Updates, Threshold and Recursive Training (Module 4), Integrator template designing (Module 6) Documentation and README

Conclusions

This project re-implements the core ideas of AutORAN as an open-source, modular, and codebase-agnostic framework for intent-driven xApp development. It introduces a blueprint-driven pipeline where each module incrementally refines a shared structured state, ensuring consistency and traceability. Although currently built on FlexRIC, the design is extensible to other RIC platforms, and leverages code-aware RAG to accurately map natural language to real system APIs for deployable outputs.

The system lowers the barrier to xApp development by enabling non-experts to generate validated, deployable applications from high-level intents. Its modular design aligns with real engineering roles, making it intuitive, maintainable, and explainable. The use of a user-validated shared blueprint reduces LLM hallucinations, offering a more reliable and scalable approach to automating network application development.

Future Work

Develop a deployment and monitoring module that can deploy xApps in a live environment, continuously analyze runtime logs, and autonomously detect and correct issues.

Integrate state-of-the-art LLM APIs to improve reasoning quality, robustness, and overall system performance.

Package the pipeline as a fully deployable end-to-end application, moving beyond a LangChain/LangGraph prototype to a production-ready system.

Enhance the RAG subsystem to reduce hallucinations and make it codebase-agnostic, enabling support for multiple RAN frameworks beyond FlexRIC.

Evaluate the system on more complex and diverse intents to uncover edge cases, latent bugs, and failure modes.

Introduce a formal code verification stage (e.g., SonarQube-style analysis) after Modules 3–6 to ensure code quality, correctness, and security.

References

- GitHub Repo Link:

<https://github.com/mohitpatil30501/Intent-Driven-Autonomous-ORAN-xApp-Development-using-Agent-AI>

- OAI [<https://gitlab.eurecom.fr/oai/openairinterface5g>]
- Flexric [<https://gitlab.eurecom.fr/mosaic5g/flexric>]

- Langchain
[<https://academy.langchain.com/courses/foundation-introduction-to-langchain-python>]
- Ollama [<https://docs.ollama.com/>]
- Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework
[<https://dl.acm.org/doi/10.1145/3718958.3750537>]
- AutORAN: LLM-driven Natural Language Programming for Agile xApp Development
[<https://arxiv.org/pdf/2603.18604>]

Declaration on LLM Usage

LLMs Leveraged

We used a mix of open-source and commercial models to balance performance and flexibility:

Gemma 4 (31B) via Ollama for local, cost-effective inference testing, GPT-OSS 120B as the primary backend LLM, Claude for script generation and AI help.

Sample Prompts Used

Intent Generation:

“Generate 5 well-defined O-RAN xApp intents covering monitoring, orchestration, optimization, and anomaly detection. Each should specify telemetry, actions, and objectives.”

RAG Evaluation:

“Given a query and retrieved code snippets, evaluate whether the Structural RAG output correctly maps to valid APIs and avoids hallucinated identifiers.”

Technical Mapping:

Modify the following script to restrict the tool to not use heredoc python style in terminal

Validation of LLM-Generated Scripts

Executed generated scripts on small-scale or synthetic datasets to verify correctness.

Performed schema validation and sanity checks (e.g., column alignment, expected outputs).

Iteratively refined outputs through module-level testing before full pipeline integration.

Manual Verification of Outputs

Documentation of Generated Code

Used GitHub versioning and tagging to track iterations and improvements.

Leveraged the structured blueprint (JSON schema) as a self-documenting artifact, where each module's output captures reasoning, intermediate states, and final decisions.

Maintained logs and artifacts in the workspace for traceability and reproducibility.